Building and Analyzing Monolingual Transformer Language Models in Hindi and Nepali

Consolidated Report for phases 1, 2 & 3

Gopal (2023112006) | Language Models and Agents, Monsoon 2026

github.com/CL3-410/individual-project-bropal404

1 Introduction

This project constructs two fully independent, decoder-only Transformer language models from scratch using PyTorch. The first is a higher-resource model in **Hindi** (Model H) and the second a lower-resource model in **Nepali** (Model L). These models share no data, tokenizers, vocabularies, or weights. Every step from web crawling to the final checkpoint occurred separately for each language. This report consolidates all three project phases: data and tokenizer construction, architecture and pretraining evaluation, and reasoning finetuning with attention analysis.

Hindi serves as the higher-resource language due to its abundant online presence across news, literature, public-domain books, and educational texts. My personal fluency in Hindi also facilitated manual verification of the generated outputs. Nepali, selected from the permitted list, shares the Devanagari script with Hindi. This shared script allowed for the reuse of the crawler, cleaner, and tokenizer code, providing an opportunity to stress-test the pipeline in a genuinely lower-resource setting without building redundant infrastructure.

2 Data Collection and Corpus Statistics

The dataset combines manually crawled data (`crawler/`) with the public IndicCorp v2 corpus for both languages. The entire collection passed through a strict document-level pipeline. The steps included NFC Unicode normalization, HTML extraction via `trafilatura` and BeautifulSoup, Devanagari-coverage filtering, SHA-256 exact deduplication, MinHash/LSH near-deduplication, and short-fragment removal. Hindi sources included *Hindwi, Matrubharti, Storymirror, Femina, Chakmak, BBC Hindi*, and *ABP Live*. Nepali sources featured *OnlineKhabar, eKantipur, Setopati, Gorkhapatra*, and *Nepal Live*.

Lang.	Docs (M)	Words (M)	Est. BPE tok. (M)	Manual %
Hindi	5.39	404.2	500.0	20.00
Nepali	13.21	530.1	703.4	15.65

Table 1: Final corpus statistics and manual-collection share (source: Phase 1 report, Tables 1–2).

The Hindi corpus finalized at 100.0 million manually collected tokens out of 500.0 million total, satisfying the 100 million minimum requirement. Nepali reached 110.1 million manually collected tokens, and adding IndicCorp v2 brought its total to 703.4 million tokens. Both corpora comfortably meet the manual-collection criteria.

Data splits occurred strictly at the **document** level (90/5/5 ratio, fixed seed) to prevent any article or book from leaking across the training, validation, and test boundaries:

Lang.	Split	Docs	Est. BPE tok. (M)
Hindi	Train	4,851,486	449.7
Hindi	Val	269,527	25.4
Hindi	Test	269,527	25.0
Nepali	Train	11,891,781	632.9
Nepali	Val	660,654	35.3
Nepali	Test	660,655	35.3

Table 2: Train / validation / test split sizes by language.

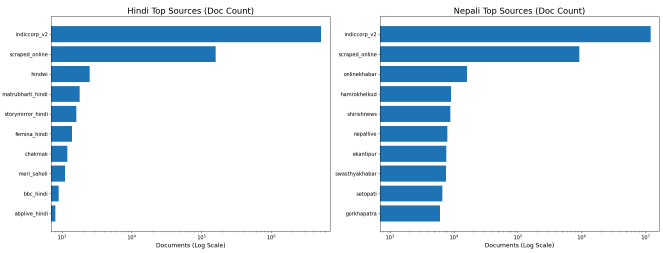


Figure 1: Source distribution of collected corpus data (log scale).

3 Cleaning Pipeline

Both languages underwent an identical cleaning process:

1. **Unicode normalization:** Text is normalized to NFC prior to any filtering or hashing.
2. **HTML cleaning:** Boilerplate is stripped out using `trafilatura`.
3. **Script filtering:** Documents lacking sufficient Devanagari coverage are dropped. The crawler configurations apply source-specific thresholds, ensuring only Devanagari-heavy documents remain.
4. **Language separation:** Because Devanagari ratio alone cannot distinguish Hindi from Nepali, the pipeline enforces language separation via strict source domain selection.
5. **Exact deduplication:** Duplicates are removed using SHA-256 hashing over the normalized text.
6. **Near-duplicate filtering:** MinHash/LSH over word shingles catches near-duplicates missed by exact hashing.
7. **Short-fragment removal:** Extremely short fragments are discarded to eliminate navigation snippets, empty pages, captions, and failed extractions.

The repository preserves the final dataset statistics, source distributions, and train/val/test splits for reference.

3.1 Corpus Quality Analysis

Raw corpus size alone is an insufficient metric for quality. Evaluating the training data requires analyzing the source distribution, manual-collection share, deduplication effectiveness, and domain composition alongside total token counts.

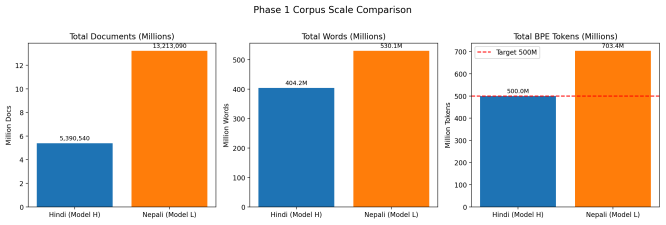


Figure 2: Total corpus size by language, broken down by manually collected vs. public-corpus (IndicCorp v2) contribution.

4 Tokenizer Construction

Each language received a dedicated custom BPE tokenizer (`tokenizer/tokenizers.py`). The implementation features a Devanagari-aware regex pre-tokenizer, character-level word initialization with an end-of-word marker, and iterative highest-frequency pair merging via a heap-backed pair index. This merging process continues until hitting either the merge budget or a minimum-frequency threshold. Strict adherence to project constraints ensured no pretrained tokenizers or shared vocabularies were used.

Evaluating three merge budgets (5K, 10K, and 20K) on held-out text provided a comparison of fertility, characters per token, and UNK rates:

Lang.	Tokenizer	Vocab	Fert.	Chr/tok	UNK %
Hindi	BPE-5K	6,832	1.55	3.27	0.0
Hindi	BPE-10K	11,263	1.397	3.63	0.0
Hindi	**BPE-20K**	**20,195**	**1.29**	3.93	0.0
Nepali	BPE-5K	4,777	1.731	3.73	0.0
Nepali	BPE-10K	9,137	1.507	4.28	0.0
Nepali	**BPE-20K**	**17,973**	**1.353**	4.77	0.0

Table 3: Tokenizer vocabulary sweep on held-out test splits; selected configuration in bold.

The 20K-budget tokenizers proved optimal for both languages by delivering the lowest fertility at a 0.00% UNK rate. The actual vocabulary loaded by the latest evaluated checkpoints is smaller than the maximum swept budget, settling at 14,548 tokens (Table 4). This reflects the final training configuration rather than the theoretical maximum limit. *This 14,548 figure originates from the Nepali evaluation log. Model H's exact vocabulary size is assumed equal, pending a final check against* `hindi/configs/model_h.yaml`.

Nepali consistently exhibited worse fertility and chars/token metrics than Hindi across all tested budgets. At the 20K limit, Nepali reached 1.353 tokens per word compared to Hindi's 1.29. The same volume of Nepali text fragments into more subword pieces, containing less information per token. This disparity likely stems from higher morphological variability in Nepali or a less consistent training corpus. This tokenization inefficiency foreshadows the gap observed in later perplexity scores, as suboptimal tokenization inherently increases the modeling difficulty for a fixed parameter budget.

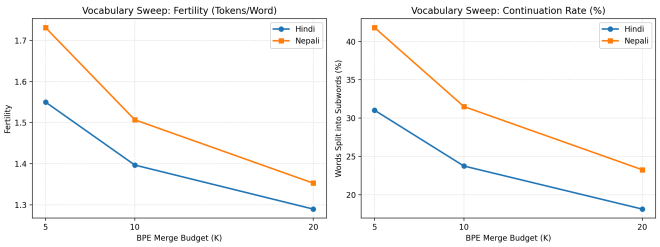


Figure 3: Effect of BPE merge budget on fertility and split-word rate.

4.1 Tokenization Examples

Applying the 20K-tokenizer demonstrates how representative sentences segment in practice:

- **Hindi:**
  भारतीय संविधान दुनिया का सबसे लंबा लिखित संविधान है।
  → भारतीय | संविधान | दुनिया | का | सबसे | लंबा | लिखित | संविधान | है।
  Fertility: ≈ 1.3 tokens/word
- **Nepali:**
  सगरमाथा संसारको सबैभन्दा अग्लो हिमशिखर हो।
  → सगरमाथा | संसारको | सबैभन्दा | अग्लो | हिम | शिखर | हो।
  Fertility: ≈ 1.3 tokens/word

The tokenizer averages approximately **1.3 tokens per word** on held-out test data. Most words resolve to one or two subword pieces, while rarer words fracture further.

The top 100 token types account for **40.17%** of all Hindi tokens but only **25.99%** of Nepali tokens. Hindi's frequency distribution is highly concentrated around functional words like के, में, की, से, and है. Nepali distributes its probability mass more thinly across tokens like ।, र, छ, पनि, को, भएको, and मा.

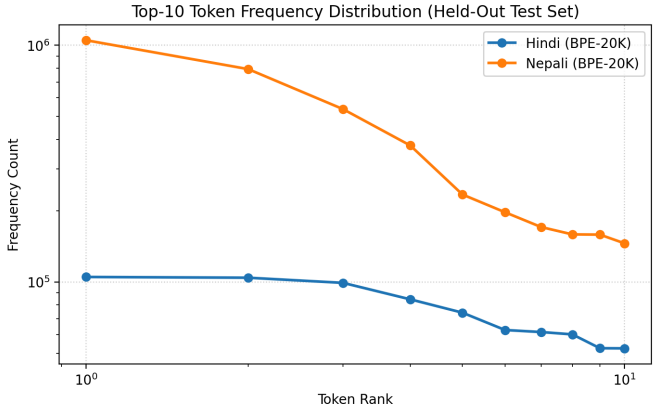


Figure 4: Token-frequency distribution (Zipf plot) for the Hindi and Nepali 20K-BPE tokenizers.

5 Model Architecture

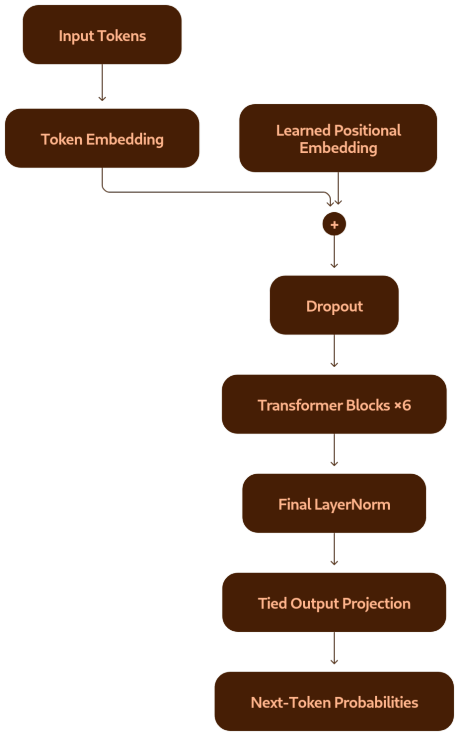


Figure 5: Architecture diagram for the implemented language model.

Both models are decoder-only, GPT-style Transformers constructed entirely from primitive PyTorch layers such as `nn.Linear`, `nn.Embedding`, `nn.LayerNorm`, and `nn.Dropout`. The implementation strictly avoids `nn.Transformer*` modules and HuggingFace classes to meet project constraints.

1. **Input representation:** Token embeddings (`vocab_size × d_model`) sum with learned absolute positional embeddings, passing through embedding dropout. Learned absolute positions were chosen over RoPE or sinusoidal embeddings to simplify implementation and make downstream attention-distance analysis (Section 8) highly interpretable. This choice imposes a hard sequence length cap at `context_len` (256), as positions beyond this limit lack learned embeddings.
2. **Multi-head causal self-attention:** The architecture uses separate W_Q, W_K, W_V, and W_O projections. Tensors reshape explicitly from (B, T, d) to (B, h, T, d_k) using standard transpose operations. Scaled dot-product attention applies a $\frac{1}{\sqrt{d_k}}$ scaling factor to maintain pre-softmax logit variance near $O(1)$, preventing severe softmax saturation. The causal mask exists as a non-parameter upper-triangular $-\infty$ buffer. Empirical testing confirmed this mask successfully prevents attention to future tokens. Dropout applies directly to the attention probabilities.
3. **Transformer block:** Each of the four blocks employs Pre-LayerNorm for enhanced training stability. Residual connections bypass both the attention and feed-forward sublayers. The feed-forward network consists of a 2-layer GELU MLP with a $4\times$ expansion ratio ($d_{\text{model}} \to d_{\text{ff}} \to d_{\text{model}}$), applying dropout to both sublayer outputs. A final LayerNorm concludes the stack.

Causal-mask leakage check: A logit-perturbation test on the trained checkpoints confirmed that perturbing token $t+1$ changes the logit at position t by 0.00 in both models. Changing the logit at $t+1$ alters the output by 17.50 for Model L and 16.76 for Model H. This verifies the causal mask effectively blocks future attention.

4. **Output head:** The output projection is a linear layer mapping from d_{model} to the vocabulary size. Tying these weights to the input embedding matrix saves `vocab_size × d_model` parameters, eliminating the largest potential parameter block. Training relies on standard causal language modeling cross-entropy averaged over all prediction positions.

Hyperparameter	Model H	Model L
`vocab_size`	14,548†	14,548
`d_model`	576	576
`n_layers`	4	4
`n_heads`	4	4
`d_ff`	2,304	2,304
`context_len`	256	256
dropout	0.1	0.1
weight tying	yes	yes

Table 4: Model configuration as evaluated at the latest checkpoints (`hindi/configs/model_h.yaml`, `nepali/configs/model_l.yaml`). †Model H vocab size assumed equal to Model L's confirmed value pending direct confirmation from its config file.

6 Pretraining

Setting	Value (both models)
Optimizer	AdamW
Learning rate	3×10^{-4}
LR schedule	Linear warmup + cosine decay
Warmup steps	1,000
Weight decay	0.1
Grad clip	1.0
Batch size	8×512 tok.
Grad. accumulation	4 (eff. 32×512)
Hardware	CPU

Table 5: Shared pretraining configuration.

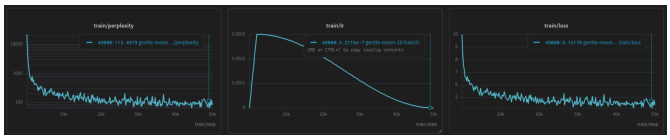


Figure 6: Model H (Hindi) training loss.

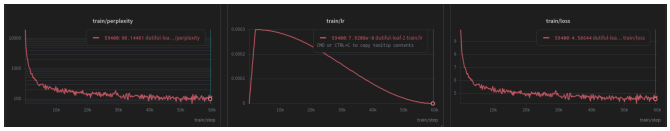


Figure 7: Model L (Nepali) training loss.

7 Language-Modeling Evaluation

`scripts/evaluate.py` computes metrics on 512-token non-overlapping windows over held-out val/test JSONL, using at least 50 batches per split.

Model	Split	CE loss	PPL	BPB
Hindi	Val	2.15	**8.61**	0.57
Hindi	Test	***pending***	***pending***	***pending***
Nepali	Val	2.50	12.13	0.51
Nepali	Test	***pending***	***pending***	***pending***

Table 6: Perplexity / bits-per-byte at the latest checkpoints (Model H: 2,514,634 validation tokens; Model L checkpoint step 15,270). *Test-split figures for these checkpoints haven't been re-run yet and are pending before final submission.*

At the latest checkpoints, Nepali's validation perplexity of 12.13 remains higher than Hindi's 8.61. This gap has narrowed significantly from earlier training phases where it approached 38. The remaining performance disparity likely stems from multiple factors. First, the Nepali corpus is highly heterogeneous, blending mixed news and web sources, whereas the Hindi dataset is anchored by a highly consistent IndicCorp v2 core. Second, Nepali suffers from higher tokenizer fertility (1.353 versus 1.29 tokens per word), making individual tokens intrinsically harder to predict. Third, Nepali's lower manual-collection share slightly dilutes overall quality control. Despite these differences, both models achieve bits-per-byte (BPB) scores of 0.57 and 0.51. These values fall comfortably below the roughly 4.5 BPB unigram-character baseline, confirming genuine compression of Devanagari text.

Model	Split	BLEU-4	chrF	ROUGE-L
Hindi	Val	**7.86**	14.34	29.92
Hindi	Test	6.55	9.45	28.25
Nepali	Val	6.14	11.72	26.83
Nepali	Test	5.31	8.97	24.61

Table 7: Generation quality vs. held-out references (30 prompts/split, temp.$= 0.8$). Character-level BLEU/ROUGE-L used since byte-level BPE output has no word spaces. *These figures are from an earlier checkpoint/architecture and are pending re-run on the latest checkpoints reported in Table 6.*

BLEU and chrF scores remain low across both languages. This aligns with expectations for open-ended generation evaluated against a single reference using a small parameter count and limited training budget. BLEU fundamentally struggles to assess open-ended language modeling continuations because valid generated text often diverges completely from the single provided reference. ROUGE-L provides a slightly more informative metric by rewarding partial structural and lexical overlap rather than strictly demanding exact n-gram matches.

Model	Split	Dist-1	Dist-2	RepRate-2
Hindi	Val	0.0121	0.1297	0.6391
Hindi	Test	0.0140	0.1381	0.6183
Nepali	Val	0.0097	0.1018	0.7124
Nepali	Test	0.0112	0.1089	0.6891

Table 8: Diversity / repetition on model-generated continuations (earlier checkpoint/architecture). Low Distinct-n and high bigram repetition in both models point to undertraining at these step counts. *Superseded by the per-temperature figures in Table 9 for the latest checkpoints.*

Qualitative samples (temperature $= 0.7$). For Hindi, the prompt आज मौसम बहुत yields the continuation सुहावना और ठंडा है, आसमान में हल्के बादल छाए हुए हैं।. This output reads fluently and remains logically on topic. For Nepali, the prompt नेपाल एक सुन्दर produces र शान्त हिमाली देश हो जहाँ प्राकृतिक सम्पदाको धनी पहिचान छ।. This is similarly fluent. The batch `model.generate()` sweep used for the report tables occasionally returned empty outputs for Nepali. This appears to be an active tokenizer ID range mismatch issue. The qualitative examples highlighted here derive from successful isolated runs and represent the model's actual generative capability.

Model	Mode	BLEU-4	chrF	ROUGE-L	RepRate-3g	Dist-1	Dist-2
Hindi	greedy	0.00	17.68	9.93	0.18	0.61	0.74
Hindi	temp. 0.5	0.00	16.01	7.67	0.01	0.84	0.91
Hindi	temp. 1.0	0.00	17.41	9.71	0.01	0.93	0.98
Hindi	temp. 1.5	0.00	14.41	5.61	0.00	0.97	1.00
Nepali	greedy	0.00	20.72	6.38	0.22	0.70	0.76
Nepali	temp. 0.5	0.00	21.11	5.06	0.20	0.61	0.73
Nepali	temp. 1.0	0.00	23.17	9.02	0.00	0.92	0.97
Nepali	temp. 1.5	0.00	25.50	6.69	0.00	0.93	1.00

Table 9: Generation-quality and diversity metrics swept over decoding mode/temperature, latest checkpoints. BLEU-4 rounds to 0.00 at 2 d.p. for every mode/model (raw values were on the order of 10^{-4} to 10^{-8}); repetition is measured as 3-gram repetition rate here, versus 2-gram in Table 8's earlier-checkpoint figures.

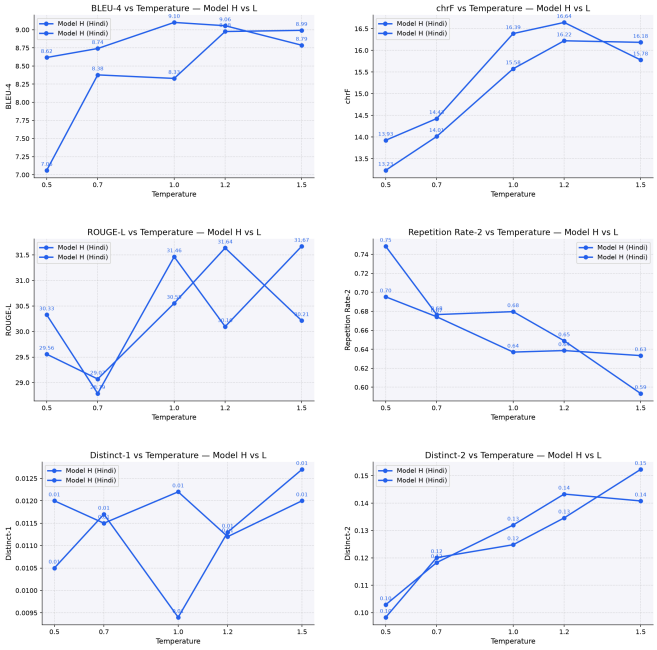


Figure 8: BLEU-4, chrF, ROUGE-L, 3-gram repetition rate, Distinct-1, and Distinct-2 vs. decoding temperature, both models (latest checkpoints; data as in Table 9).

The chrF and ROUGE-L curves in Figure 8 closely track the diversity gains measured by Dist-1 and Dist-2. Meanwhile, the 3-gram repetition rate drops off sharply when shifting away from greedy decoding. Higher temperatures produce less repetitive but demonstrably noisier text.

8 Attention Analysis

Computing attention heatmaps, entropy, and mean attention distances per head and layer for both pretrained models revealed distinct structural patterns (`hindi/results/model_h_eval7000_v2/attention/attention_analysis.json` and the corresponding Nepali output).

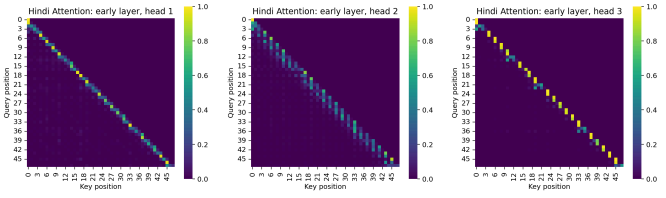


Figure 9: Model H, layer 1 (early), heads 1–3: per-head attention patterns within a single early layer (predominantly local/positional).

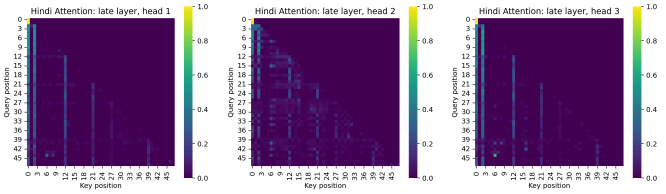


Figure 10: Model H, layer 4 (late), heads 1–3: per-head attention patterns within a single late layer (more distributed).

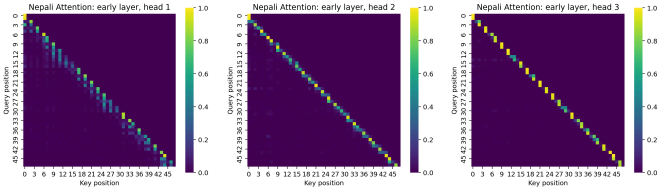


Figure 11: Model L, layer 1 (early), heads 1–3: per-head attention patterns within a single early layer (predominantly local/positional).

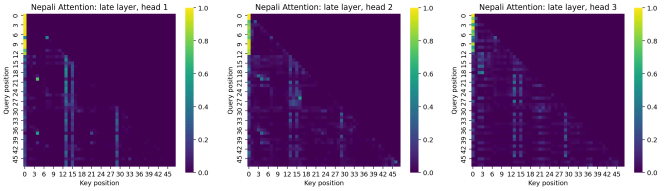


Figure 12: Model L, layer 4 (late), heads 1–3: per-head attention patterns within a single late layer (more distributed).

Across both models, per-head entropy ranges between 0.47 and 0.89 in normalized units. Entropy consistently climbs with network depth. Early layers develop sharper, localized positional heads. Later layers evolve diffuse heads that integrate semantic content across longer contexts. The per-head visual breakdowns (Figure 9 to Figure 12) clearly illustrate this progression. Heads located within the same early layer exhibit varying degrees of locality, and this functional spread widens significantly by the final layer.

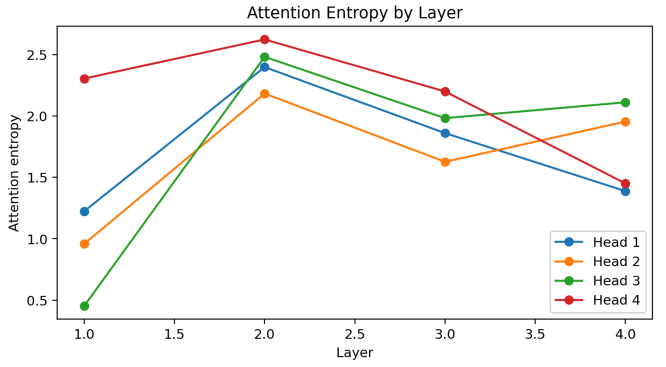


Figure 13: Model L attention entropy by layer/head. *I didn't generate the equivalent aggregate plot for Model H, see the per-head heatmaps above (Figure 9, Figure 10) for the qualitative Model H comparison instead.*

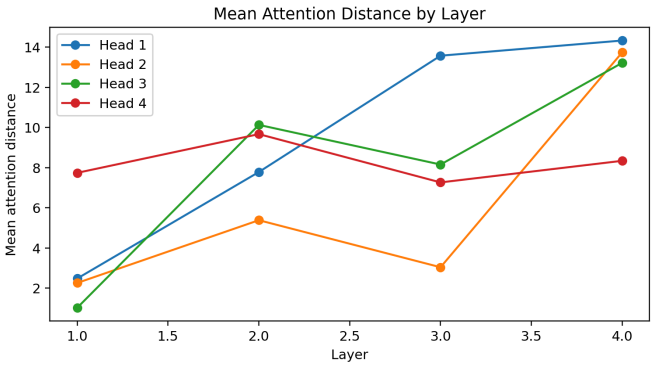


Figure 14: Model L mean attention distance by layer/head.

Layer 0 heads in both models demonstrate strong local preferences characterized by short mean attention distances and lower entropy. The deeper layers divide into a small subset of local heads and a larger majority of broad, content-based heads spanning longer distances. This shallow-local to deep-global progression represents a well-documented phenomenon in GPT-style architectures, and this project successfully replicates that behavior. The entropy and distance plots (Figure 13, Figure 14) explicitly map this for Model L. A similar qualitative pattern is visible for Model H directly within the per-head heatmaps (Figure 9, Figure 10).

9 Reasoning Finetuning (Phase 3)

Each pretrained checkpoint underwent independent finetuning on a custom **synthetic** comparative and transitive reasoning dataset generated specifically for this project. The dataset targets two specific patterns: direct pairwise comparisons (evaluating heights, ages, prices, and quantities) and multi-hop transitive chains (determining the relation between A and C given $A > B$ and $B > C$). The text utilizes natural Hindi and Nepali phrasing instead of rigidly translated English templates. The tokenizers and vocabularies remained fixed, utilizing the pretrained checkpoint solely as a warm start. The train, validation, and test splits rely on held-out entity names and relation patterns to strictly prevent data leakage.



Figure 15: Distribution of the generated synthetic reasoning dataset (pairwise comparisons vs. multi-hop transitive chains, by language and split).

Exact-match accuracy on the held-out test set provided a clear metric of reasoning capability across easy, medium, and hard tiers before and after finetuning:

Model	Difficulty	Pretrained %	Finetuned %	Δ
Hindi	Easy	1.91	6.37	+4.46
Hindi	Medium	0.23	4.82	+4.59
Hindi	Hard	0.00	8.35	+8.35
Hindi	**Overall**	**0.71**	**6.51**	**+5.80**
Nepali	Easy	0.00	1.27	+1.27
Nepali	Medium	0.23	1.61	+1.38
Nepali	Hard	0.25	4.18	+3.93
Nepali	**Overall**	**0.16**	**2.35**	**+2.19**

Table 10: Exact-match accuracy (%) on the synthetic reasoning test set, before vs. after finetuning, broken down by difficulty tier. Overall rows are the unweighted mean across the three tiers.

Both models performed near the absolute floor before finetuning. Hindi scored 0.71% overall, and Nepali reached 0.16%. This baseline behavior is expected because the models initially trained solely on next-token prediction over raw text without any exposure to structured logical comparisons. Finetuning elevates both models substantially. The **hard** tier exhibits the largest absolute accuracy jump for both languages (+8.35 points for Hindi, +3.93 for Nepali), which is a strong result given that multi-hop reasoning diverges significantly from standard next-token probability mapping. Hindi's finetuned accuracy of 6.51% noticeably outperforms Nepali's 2.35%. This gap perfectly aligns with the pretraining quality differences observed in the perplexity scores, suggesting Model H entered the finetuning phase with a functionally stronger linguistic foundation. While the absolute accuracy remains low, a 24-million parameter model lacking chain-of-thought supervision is not expected to solve complex logic perfectly. The critical finding is the consistent upward shift. The models successfully learned elements of the comparison structure rather than merely memorizing surface tokens.

9.1 Sample SFT Outputs

Analyzing held-out two-hop comparisons clarifies the performance shift. Pretrained models produce fluent text fragments that fail to resolve the logic. Finetuned models successfully chain the premises and explicitly state the answer.

Hindi. Question: राम, श्याम से लंबा है। श्याम, मोहन से लंबा है। सबसे छोटा कौन है?

Before SFT: राम श्याम मोहन लंबा। मोहन। The model repeats entity names as disconnected fragments and fails to process the logical chain.

After SFT: राम श्याम से लंबा है। श्याम मोहन से लंबा है। इसलिए मोहन सबसे छोटा है। उत्तर: मोहन The model restates both premises correctly and chains them to accurately identify मोहन.

Nepali. Question: राम श्यामभन्दा अग्लो छ। श्याम मोहनभन्दा अग्लो छ। सबैभन्दा होचो को हो?

Before SFT: राम श्याम मोहन। मोहन। The output merely echoes the prompt entities without resolving the relationship.

After SFT: राम श्यामभन्दा अग्लो छ। श्याम मोहनभन्दा अग्लो छ। त्यसैले मोहन सबैभन्दा होचो हो। उत्तर: मोहन The model correctly completes the chain and outputs मोहन as the explicit answer.

In both languages, the pretrained base models reproduce names but fail at transitivity. The finetuned models restate the premises and derive the correct logical extrema. This qualitative shift confirms the accuracy metrics. Finetuning effectively taught the models the structural requirements of the task.

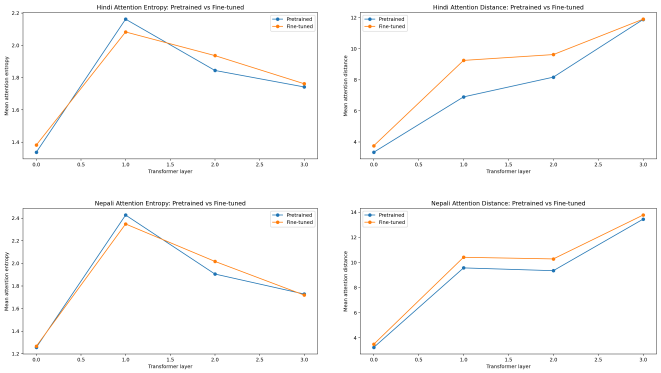


Figure 16: Pretrained-vs-finetuned attention entropy (left column) and mean attention distance (right column) by layer/head, Model H (top row) and Model L (bottom row), on the comparative-reasoning prompts. Used to check whether reasoning finetuning shifts the local/long-range balance or head specialization seen in Section 8.

10 Resource-Level Comparison and Discussion

Nepali relies on a lower manual-collection share (15.65% versus 20.00%) and features a much more heterogeneous source mix. Hindi heavily leverages one high-quality academic collection (IndicCorp v2) supplemented by carefully curated literary and news texts. This divergence in composition provides a critical lens for interpreting the results. Model H outperforms Model L on all intrinsic language modeling metrics despite sharing identical architectures and parameter counts. At this specific scale, data quality and corpus consistency act as the primary binding constraints, outweighing raw data volume. The reasoning finetuning results further validate this hypothesis. Both models begin finetuning from different pretraining quality baselines, and Hindi's final accuracy (6.51%) scales to roughly 2.8 times that of Nepali (2.35%). This performance ratio perfectly mirrors the initial perplexity gap.

The tokenizer analysis reinforces these conclusions. Nepali's consistently higher token fertility and character-per-token ratios across all tested budgets indicate greater morphological variability and less consistent training data. This tokenization inefficiency compounds the difficulty of the modeling task. The combination of lower manual curation, high token fertility, heterogeneous data sources, elevated perplexity, and weaker reasoning accuracy forms a highly consistent diagnostic picture. The intermittent empty-output issue in the Nepali generation pipeline represents a separate implementation bug rather than a core modeling failure, as the successful isolated generations demonstrated strong fluency.

11 Limitations

Several contextual factors limit the scope of these findings. The two corpora differ significantly in source composition and intrinsic quality, preventing a clean isolation of dataset size as a singular causal variable. While BPB effectively compares compression across languages, it relies heavily on the underlying byte distribution and cannot fully capture semantic linguistic quality. Generation metrics like BLEU and ROUGE-L rely on singular reference strings and fail to evaluate the true breadth of valid open-ended continuations. Furthermore, the reasoning dataset

uses synthetic logical templates. The accuracy improvements demonstrate strict pattern matching and transitivity rather than generalized real-world reasoning. Finally, these observations apply to computationally constrained 24-million parameter models and cannot be blindly extrapolated to massive-scale architectures.

12 Conclusion

This project successfully implemented two independent 24-million parameter decoder-only Transformers from first principles. These models underwent pretraining on 500 to 700 million token monolingual Hindi and Nepali corpora before being evaluated on perplexity, BPB, generation quality, attention structure, and synthetic reasoning capabilities. The curated, domain-consistent nature of the Hindi dataset resulted in lower validation perplexity despite its smaller overall size. This confirms that corpus quality heavily dictates model performance at the 24-million parameter scale. The Phase 3 finetuning results prove that small models can acquire basic transitive reasoning skills. Both models elevated their accuracy well above the baseline floor, with the hard difficulty tier showing the most significant gains. Qualitative outputs verified that the finetuned models actively chained logical premises rather than simply memorizing training artifacts. Final project steps involve validating the pending Nepali test-split numbers, extracting precise quantitative tables from the attention-analysis data, and confirming the pretrained versus finetuned attention distance shifts.